

Министерство образования Республики Беларусь  
Учреждение образования  
«Брестский Государственный технический университет»  
Кафедра ИИТ

**Лабораторная работа №1**

По дисциплине «Обработка изображений в интеллектуальных системах»

Тема: «Обучение классификаторов средствами библиотеки PyTorch»

**Выполнил:**

Студент 3 курса

Группы ИИ-25

Бекназарова А. С.

**Проверил:**

Андренко К. В.

**Цель работы:** научиться конструировать нейросетевые классификаторы и выполнять их обучение на известных выборках компьютерного зрения.

**Задание:** выполнить конструирование своей модели СНС, обучить ее на выборке по заданию (использовать `torchvision.datasets`). Предпочтение отдавать как можно более простым архитектурам, базирующимся на базовых типах слоев (сверточный, полносвязный, подвыборочный, слой нелинейного преобразования). Оценить эффективность обучения на тестовой выборке, построить график изменения ошибки (`matplotlib`); Ознакомьтесь с state-of-the-art результатами для предлагаемых выборок (из материалов в сети Интернет). Сделать выводы о результатах обучения СНС из п. 1; реализовать визуализацию работы СНС из пункта 1 (выбор и подачу на архитектуру произвольного изображения с выводом результата); оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

## Вариант 2

**Датасет:** Fashion-MNIST, **Размер изображения:** 28\*28, **Оптимизатор:** SGD

## Ход работы

### Архитектура:

Три свёрточных слоя с ReLU и max-pooling, затем два полносвязных слоя:

$\text{Conv2d}(1, 32, 3, p=1) \rightarrow \text{ReLU} \rightarrow \text{Conv2d}(32, 64, 3, p=1) \rightarrow \text{ReLU} \rightarrow \text{MaxPool2d}(2) \rightarrow 64 \times 14 \times 14$

$\text{Conv2d}(64, 128, 3, p=1) \rightarrow \text{ReLU} \rightarrow \text{MaxPool2d}(2) \rightarrow 128 \times 7 \times 7$

$\text{Flatten} \rightarrow \text{Linear}(128 \cdot 7 \cdot 7, 512) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.5) \rightarrow \text{Linear}(512, 10)$

### Код:

```
class FashionCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 32, 3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, 3, padding=1)
        self.conv3 = nn.Conv2d(64, 128, 3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.dropout = nn.Dropout(0.5)
        self.fc1 = nn.Linear(128 * 7 * 7, 512)
        self.fc2 = nn.Linear(512, 10)

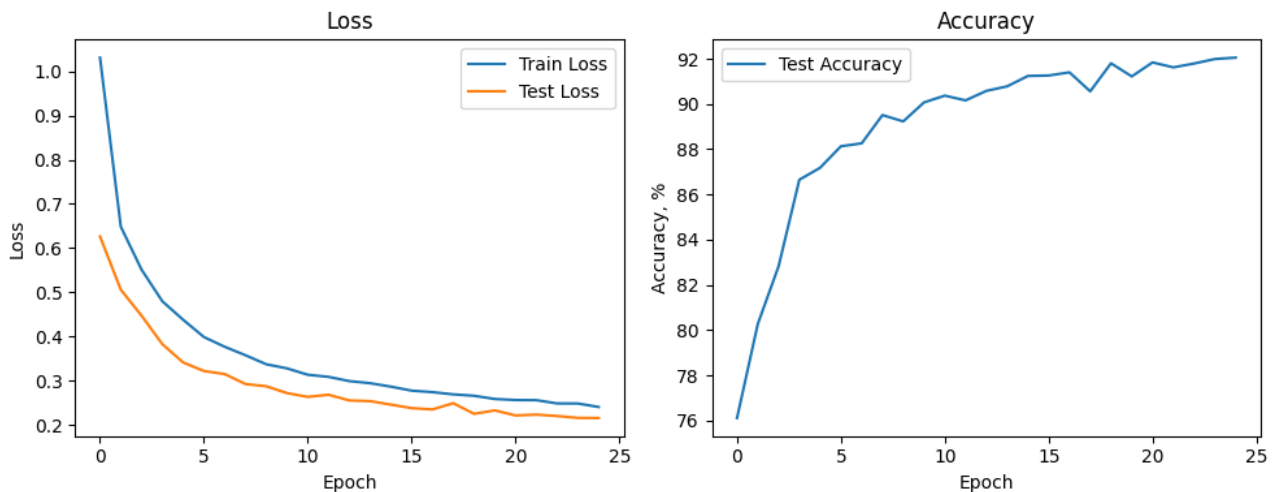
    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = F.relu(self.conv2(x))
        x = self.pool(x)
        x = F.relu(self.conv3(x))
        x = self.pool(x)
        x = x.view(x.size(0), -1)
        x = self.dropout(F.relu(self.fc1(x)))
        x = self.fc2(x)
        return x
```

```
model = FashionCNN().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=0.01,
                        momentum=0.9, weight_decay=1e-4)
```

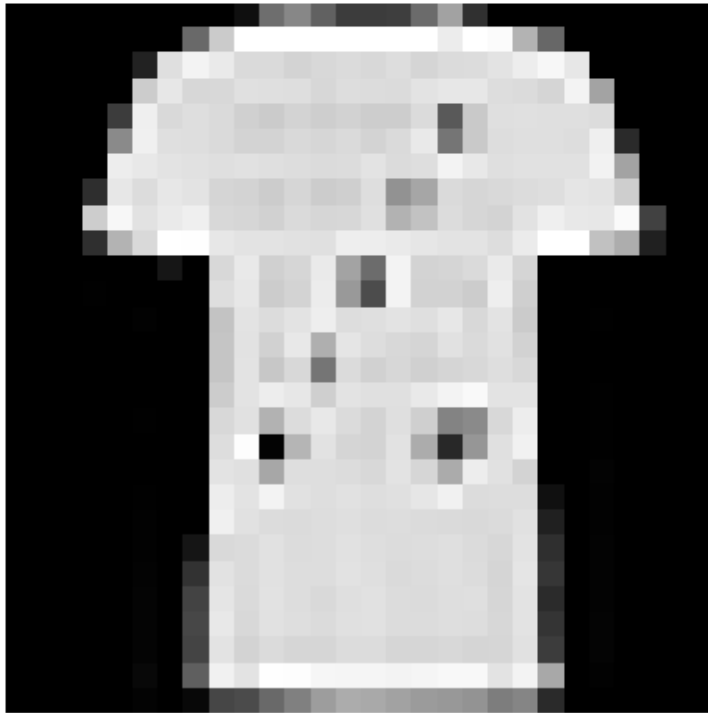
### Результаты:

|             |                    |                   |                  |
|-------------|--------------------|-------------------|------------------|
| Epoch 1/25  | Train Loss: 1.0305 | Test Loss: 0.6266 | Test Acc: 76.11% |
| Epoch 2/25  | Train Loss: 0.6487 | Test Loss: 0.5063 | Test Acc: 80.27% |
| Epoch 3/25  | Train Loss: 0.5515 | Test Loss: 0.4474 | Test Acc: 82.82% |
| Epoch 4/25  | Train Loss: 0.4801 | Test Loss: 0.3832 | Test Acc: 86.65% |
| Epoch 5/25  | Train Loss: 0.4384 | Test Loss: 0.3419 | Test Acc: 87.18% |
| Epoch 6/25  | Train Loss: 0.3993 | Test Loss: 0.3226 | Test Acc: 88.13% |
| Epoch 7/25  | Train Loss: 0.3773 | Test Loss: 0.3157 | Test Acc: 88.26% |
| Epoch 8/25  | Train Loss: 0.3582 | Test Loss: 0.2931 | Test Acc: 89.51% |
| Epoch 9/25  | Train Loss: 0.3379 | Test Loss: 0.2881 | Test Acc: 89.23% |
| Epoch 10/25 | Train Loss: 0.3285 | Test Loss: 0.2726 | Test Acc: 90.07% |
| Epoch 11/25 | Train Loss: 0.3141 | Test Loss: 0.2642 | Test Acc: 90.37% |
| Epoch 12/25 | Train Loss: 0.3093 | Test Loss: 0.2690 | Test Acc: 90.16% |
| Epoch 13/25 | Train Loss: 0.2998 | Test Loss: 0.2560 | Test Acc: 90.58% |
| Epoch 14/25 | Train Loss: 0.2951 | Test Loss: 0.2545 | Test Acc: 90.78% |
| Epoch 15/25 | Train Loss: 0.2872 | Test Loss: 0.2466 | Test Acc: 91.24% |
| Epoch 16/25 | Train Loss: 0.2782 | Test Loss: 0.2388 | Test Acc: 91.26% |
| Epoch 17/25 | Train Loss: 0.2748 | Test Loss: 0.2358 | Test Acc: 91.40% |
| Epoch 18/25 | Train Loss: 0.2698 | Test Loss: 0.2499 | Test Acc: 90.56% |
| Epoch 19/25 | Train Loss: 0.2666 | Test Loss: 0.2259 | Test Acc: 91.80% |
| Epoch 20/25 | Train Loss: 0.2594 | Test Loss: 0.2336 | Test Acc: 91.22% |
| Epoch 21/25 | Train Loss: 0.2570 | Test Loss: 0.2222 | Test Acc: 91.84% |
| Epoch 22/25 | Train Loss: 0.2567 | Test Loss: 0.2241 | Test Acc: 91.62% |
| Epoch 23/25 | Train Loss: 0.2494 | Test Loss: 0.2209 | Test Acc: 91.79% |
| Epoch 24/25 | Train Loss: 0.2492 | Test Loss: 0.2167 | Test Acc: 91.99% |
| Epoch 25/25 | Train Loss: 0.2413 | Test Loss: 0.2163 | Test Acc: 92.05% |

### Графики:



True: T-shirt/top  
Pred: T-shirt/top



**Вывод:** научились конструировать нейросетевые классификаторы и выполнять их обучение на известных выборках компьютерного зрения.